

Complete Hadoop Setup + WordCount (Ubuntu)

1. Install Java

```
sudo apt update && sudo apt install openjdk-8-jdk -y
```

2. Set JAVA_HOME

```
nano ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export PATH=$PATH:$JAVA_HOME/bin
source ~/.bashrc
```

3. Download Hadoop

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
```

4. Set Hadoop Environment

```
nano ~/.bashrc
export HADOOP_HOME=$HOME/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
source ~/.bashrc
```

5. Install SSH

```
sudo apt install openssh-server -y
sudo service ssh start
```

6. Setup SSH Key

```
ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
ssh localhost
```

7. Hadoop Configuration

```
core-site.xml:
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://localhost:9000</value>
</property>
```

```

</configuration>

hdfs-site.xml:
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
<property>
<name>dfs.name.dir</name>
<value>file:///home/$USER/hadoopdata/hdfs/namenode</value>
</property>
<property>
<name>dfs.data.dir</name>
<value>file:///home/$USER/hadoopdata/hdfs/datanode</value>
</property>
</configuration>

mapred-site.xml:
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>yarn.app.mapreduce.am.env</name>
<value>HADOOP_MAPRED_HOME=$HOME/hadoop</value>
</property>
<property>
<name>mapreduce.map.env</name>
<value>HADOOP_MAPRED_HOME=$HOME/hadoop</value>
</property>
<property>
<name>mapreduce.reduce.env</name>
<value>HADOOP_MAPRED_HOME=$HOME/hadoop</value>
</property>
</configuration>

yarn-site.xml:
<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>

```

8. Create Directories

```

mkdir -p ~/hadoopdata/hdfs/namenode
mkdir -p ~/hadoopdata/hdfs/datanode

```

9. Format Namenode

```

hdfs namenode -format

```

10. Start Hadoop

```

start-dfs.sh
start-yarn.sh
jps

```

11. WordCount Java Code

```

import java.io.IOException;

```

```

import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.*;
import org.apache.hadoop.mapreduce.lib.output.*;

public class WordCount {
    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>{
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer extends Reducer<Text,IntWritable,Text,IntWritable>{
        private IntWritable result = new IntWritable();
        public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) sum += val.get();
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

12. Compile & Run

```

mkdir wc_classes
javac -classpath `hadoop classpath` -d wc_classes WordCount.java
jar -cvf wordcount.jar -C wc_classes/ .
echo "hello hadoop hello world" > input.txt
hdfs dfs -mkdir /input
hdfs dfs -put input.txt /input
hadoop jar wordcount.jar WordCount /input /output
hdfs dfs -cat /output/part-r-00000

```